

Tipos Abstractos de Datos

Algoritmos y Estructuras de Datos

26 de Agosto de 2026

Plan del día

- Anatomía de un TAD
- Ejercicio de TADs simples
- Ustedes hacen ejercicios de TADs
- Ejercicio de TADs de parcial

¿Qué es un TAD?

Un TAD (Tipo Abstracto de Datos)

- Define un conjunto de **valores** y las **operaciones** que se pueden realizar sobre ellos.
- Es **abstracto**: se enfoca en la funcionalidad, sin importar
 - Representación interna de los datos
 - Implementación de las operaciones
- Especifica **qué** hacen las operaciones y no **cómo** lo hacen

Ejercicio 1

Especificar en forma completa el TAD NumeroRacional que incluya las operaciones aritméticas básicas (suma, resta, división, multiplicación) y la operación de *igualdad* que dados dos números racionales devuelva verdadero si son iguales.

¿Por donde empezamos?

Ejercicio 1

Especificar en forma completa el TAD NumeroRacional que incluya las operaciones aritméticas básicas (suma, resta, división, multiplicación) y la operación de *igualdad* que dados dos números racionales devuelva verdadero si son iguales.

Primero el nombre :)

```
TAD NumeroRacional {
```

```
}
```

Ejercicio 1

Especificar en forma completa el TAD NumeroRacional que incluya las operaciones aritméticas básicas (suma, resta, división, multiplicación) y la operación de *igualdad* que dados dos números racionales devuelva verdadero si son iguales.

Después las operaciones (mi recomendación)

- Nos indican qué cosas se puede hacer con una instancia del TAD
- Sólo se interactúa con el TAD a través de sus operaciones

```
TAD NumeroRacional {  
  proc suma(                                ):  
  proc resta(                               ):  
  proc división(                             ):  
  proc igualdad(                             ):  
}
```

Ejercicio 1

Especificar en forma completa el TAD NumeroRacional que incluya las operaciones aritméticas básicas (suma, resta, división, multiplicación) y la operación de *igualdad* que dados dos números racionales devuelva verdadero si son iguales.

Después las operaciones (mi recomendación)

```
TAD Racional {  
  proc suma(in r1: Racional, in r2: Racional): Racional  
  proc resta(in r1: Racional, in r2: Racional): Racional  
  proc división(in r1: Racional, in r2: Racional): Racional  
  proc igualdad(in r1: Racional, in r2: Racional): Bool  
}
```

Ejercicio 1

Especificar en forma completa el TAD NumeroRacional que incluya las operaciones aritméticas básicas (suma, resta, división, multiplicación) y la operación de *igualdad* que dados dos números racionales devuelva verdadero si son iguales.

Alternativa

```
TAD Racional {  
  proc suma(inout r1: Racional, in r2: Racional)  
  proc resta(inout r1: Racional, in r2: Racional)  
  proc división(inout r1: Racional, in r2: Racional)  
  proc igualdad(in r1: Racional, in r2: Racional): Bool  
}
```

Ejercicio 1

¿Qué necesitamos para “explicar” qué hacen las operaciones?
... es lo mismo que preguntarse ...
¿qué observadores necesitamos?

```
TAD Racional {  
  obs numerador:  $\mathbb{Z}$   
  obs denominador:  $\mathbb{Z}$   
  
  proc suma(in r1: Racional, in r2: Racional): Racional  
  proc resta(in r1: Racional, in r2: Racional): Racional  
  proc división(in r1: Racional, in r2: Racional): Racional  
  proc igualdad(in r1: Racional, in r2: Racional): Bool  
}
```

Ejercicio 1

Ahora sí explicamos las operaciones

```
TAD Racional {  
  obs numerador:  $\mathbb{Z}$   
  obs denominador:  $\mathbb{Z}$   
  
  proc suma(in r1: Racional, in r2: Racional): Racional  
    require {true}  
    asegura {res.numerador = ...  $\wedge$  res.denominador = ...}  
  
  proc resta(in r1: Racional, in r2: Racional): Racional  
    require {...}  
    asegura {...}  
}
```

Ejercicio 3c

Especifique el TAD DobleCola $\langle T \rangle$, en el que los elementos pueden insertarse al principio o al final y se eliminan por el medio. Debe contener las operaciones

- *nuevaDobleCola* que crea una cola vacía
- *estáVacía* que devuelve true si la cola no contiene elementos
- *encolarAtrás* que agrega un elemento al final de la cola
- *encolarAdelante* que agrega un elemento al principio de la cola
- *desencolar* que elimina el primer elemento de la cola y lo devuelve

Ejemplo:

```
c := NuevaDobleCola<T>()
encolarAdelante(c, 1)    // c = <1>
encolarAdelante(c, 2)    // c = <2,1>
encolarAtrás(c, 3)       // c = <2,1,3>
desencolar(c)            // devuelve 1, c = <2,3>
desencolar(c)            // devuelve 2, c = <3>
desencolar(c)            // devuelve 3, c = <>
```

Recreo

Ya pueden hacer la práctica hasta el ejercicio 9. ¿Nos vemos en un rato!

Buscaminas

El juego del **Buscaminas** consiste en un tablero rectangular dividido en casillas que pueden estar libres o tener una bomba. Al iniciar el juego todas las casillas están “cubiertas”, es decir, no se puede distinguir si tienen una bomba o no.

En cada turno hay que descubrir alguna de las casillas cubiertas. Si se descubre una bomba, reventamos y perdemos el juego. Si se descubre una casilla libre podemos seguir jugando. Para las casillas descubiertas el juego nos dice cuántas bombas hay alrededor de esa casilla, un número que puede ir del 0 al 8 aunque en una interfaz gráfica el 0 suele no mostrarse.

Cuando además de estar libre la casilla tiene 0 bombas alrededor, se descubren automáticamente todas sus casillas adyacentes. Esto puede desatar una reacción en cadena y descubrir un grupo grande de casillas en una sola jugada.

El objetivo del juego es descubrir todas las casillas libres, sin reventar en el proceso. El juego muestra el estado actual con una carita que puede ser “muerta” (perdimos) “feliz” (podemos seguir jugando) o “cool” (ganamos!)

Como ayuda se pueden poner banderines en las casillas cubiertas, para marcar que se cree que hay una bomba ahí y evitar descubrirlas. No se permite tratar de jugar una casilla que tiene banderín, pero se pueden sacar los banderines.